

Learn how Amazon Pharmacy created their LLM-based chat-bot using Amazon SageMaker

by Burak Gozluklu, Alexandre Alves, Jangwon Kim, and Nirvay Kumar | on 17 OCT 2023 | in [Amazon SageMaker](#), [Amazon SageMaker JumpStart](#), [Artificial Intelligence](#), [Generative AI](#), [Healthcare](#) | [Permalink](#) | [Comments](#) | [Share](#)

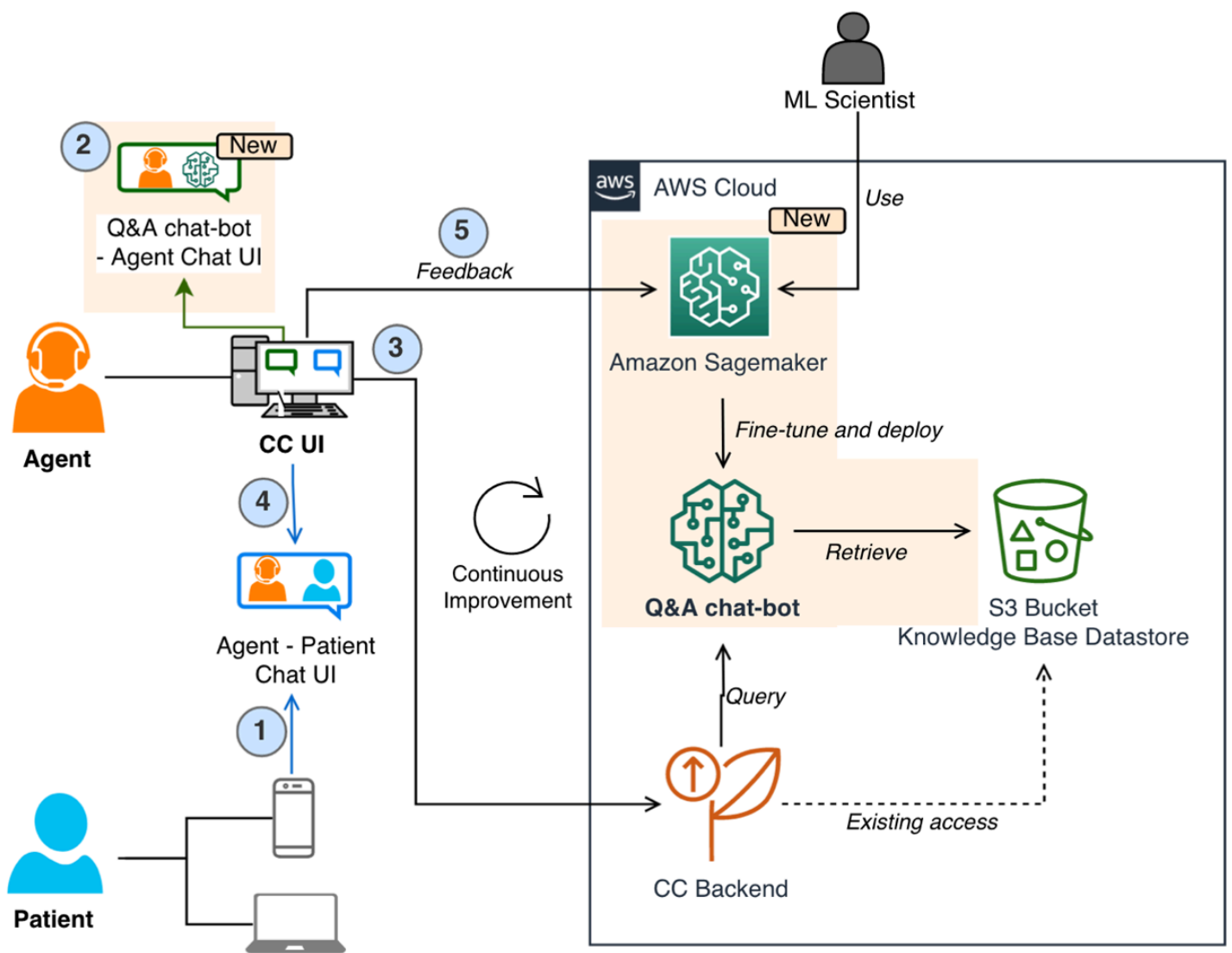
[Amazon Pharmacy](#) is a full-service pharmacy on [Amazon.com](#) that offers transparent pricing, clinical and customer support, and free delivery right to your door. Customer care agents play a crucial role in quickly and accurately retrieving information related to pharmacy information, including prescription clarifications and transfer status, order and dispensing details, and patient profile information, in real time. Amazon Pharmacy provides a chat interface where customers (patients and doctors) can talk online with customer care representatives (agents). One challenge that agents face is finding the precise information when answering customers' questions, because the diversity, volume, and complexity of healthcare's processes (such as explaining prior authorizations) can be daunting. Finding the right information, summarizing it, and explaining it takes time, slowing down the speed to serve patients.

To tackle this challenge, Amazon Pharmacy built a [generative AI](#) question and answering (Q&A) chatbot assistant to empower agents to retrieve information with natural language searches in real time, while preserving the human interaction with customers. The solution is HIPAA compliant, ensuring customer privacy. In addition, agents submit their feedback related to the machine-generated answers back to the Amazon Pharmacy development team, so that it can be used for future model improvements.

In this post, we describe how Amazon Pharmacy implemented its customer care agent assistant chatbot solution using AWS AI products, including foundation models in [Amazon SageMaker JumpStart](#) to accelerate its development. We start by highlighting the overall experience of the customer care agent with the addition of the large language model (LLM)-based chatbot. Then we explain how the solution uses the Retrieval Augmented Generation (RAG) pattern for its implementation. Finally, we describe the product architecture. This post demonstrates how generative AI is integrated into an already working application in a complex and highly regulated business, improving the customer care experience for pharmacy patients.

The LLM-based Q&A chatbot

The following figure shows the process flow of a patient contacting Amazon Pharmacy customer care via chat (Step 1). Agents use a separate internal customer care UI to ask questions to the LLM-based Q&A chatbot (Step 2). The customer care UI then sends the request to a service backend hosted on [AWS Fargate](#) (Step 3), where the queries are orchestrated through a combination of models and data retrieval processes, collectively known as the RAG process. This process is the heart of the LLM-based chatbot solution and its details are explained in the next section. At the end of this process, the machine-generated response is returned to the agent, who can review the answer before providing it back to the end-customer (Step 4). It should be noted that agents are trained to exercise judgment and use the LLM-based chatbot solution as a tool that augments their work, so they can dedicate their time to personal interactions with the customer. Agents also label the machine-generated response with their feedback (for example, positive or negative). This feedback is then used by the Amazon Pharmacy development team to improve the solution (through fine-tuning or data improvements), forming a continuous cycle of product development with the user (Step 5).



The following figure shows an example from a Q&A chatbot and agent interaction. Here, the agent was asking about a claim rejection code. The Q&A chatbot (Agent AI Assistant) answers the question with a clear description of the rejection code. It also provides the link to the original documentation for the agents to follow up, if needed.

When posting questions to the assistant, please do not enter PHI

What's a claim rejection code of 11?

Agent AI Assistant | RequestId: 32598c1e-5feb-41e3-9115-ad9565e74d3e 

The patient relationship code we have is missing or incomplete. Please double check your insurance card and ensure your patient relationship code is entered accurately.

References:

- [Rejection Code: 11](#)

Snippet 1: Rejection Code: 11

Rejection Title: M/I Patient Relationship Code

Rejection Description: Missing relationship code...

[Show more](#)

Accelerating the ML model development

In the previous figure depicting the chatbot workflow, we skipped the details of how to train the initial version of the Q&A chatbot models. To do this, the Amazon Pharmacy development team benefited from using SageMaker JumpStart. SageMaker JumpStart allowed the team to experiment quickly with different models, running different benchmarks and tests, failing fast as needed. *Failing fast* is a concept practiced by the scientist and developers to quickly build solutions as realistic as possible and learn from their efforts to make it better in the next iteration. After the team decided on the model and performed any necessary fine-tuning and customization, they used [SageMaker hosting](#) to deploy the solution. The reuse of the foundation models in SageMaker JumpStart allowed the development team to cut months of work that otherwise would have been needed to train models from scratch.

The RAG design pattern

One core part of the solution is the use of the [Retrieval Augmented Generation](#) (RAG) design pattern for implementing Q&A solutions. The first step in this pattern is to identify a set of known question and answer pairs, which is the initial ground truth for the solution. The next step is to convert the questions to a better representation for the purpose of similarity and searching, which is called *embedding* (we embed a higher-dimensional object into a hyperplane with less dimensions). This is done through an embedding-specific foundation model. These embeddings are used as indexes to the answers, much like how a database index maps a primary key to a row. We're now ready to support new queries coming from the customer. As explained previously, the experience is that customers send their queries to agents, who then interface with the LLM-based chatbot. Within the Q&A chatbot, the query is converted to an embedding and then used as a search key for a matching index (from the previous step). The matching criteria is based on a similarity model, such as [FAISS](#) or [Amazon Open Search Service](#) (for more details, refer to [Amazon OpenSearch Service's vector database capabilities explained](#)). When there are matches, the top answers are retrieved and used as the prompt context for the generative model. This corresponds to the second step in the RAG pattern—the generative step. In this step, the prompt is sent to the LLM (generator foundation model), which composes the final machine-generated response to the original question. This response is provided back through the customer care UI to the agent, who validates the answer, edits it if needed, and sends it back to the patient. The following diagram illustrates this process.

Managing the knowledge base

As we learned with the RAG pattern, the first step in performing Q&A consists of retrieving the data (the question and answer pairs) to be used as context for the LLM prompt. This data is referred to as the chatbot's *knowledge base*. Examples of this data are Amazon Pharmacy internal standard operating procedures (SOPs) and information available in [Amazon Pharmacy Help Center](#). To facilitate the indexing and the retrieval process (as described previously), it's often useful to gather all this information, which may be hosted across different solutions such as in wikis, files, and databases, into a single repository. In the particular case of the Amazon Pharmacy chatbot, we use [Amazon Simple Storage Service](#) (Amazon S3) for this purpose because of its simplicity and flexibility.

Solution overview

The following figure shows the solution architecture. The customer care application and the LLM-based Q&A chatbot are deployed in their own VPC for network isolation. The connection between the [VPC endpoints](#) is realized through [AWS PrivateLink](#), guaranteeing their privacy. The Q&A chatbot likewise has its own AWS account for role separation, isolation, and ease of monitoring for security, cost, and compliance purposes. The Q&A chatbot orchestration logic is hosted in Fargate with [Amazon Elastic Container Service](#) (Amazon ECS). To set up PrivateLink, a [Network Load Balancer](#) proxies the requests to an [Application Load Balancer](#), which stops the end-client TLS connection and hands requests off to Fargate. The primary storage service is Amazon S3. As mentioned previously, the related input data is imported into the desired format inside the Q&A chatbot account and persisted in S3 buckets.

When it comes to the machine learning (ML) infrastructure, [Amazon SageMaker](#) is at the center of the architecture. As explained in the previous sections, two models are used, the embedding model and the LLM model, and these are hosted in two separate SageMaker endpoints. By using the SageMaker [data capture](#) feature, we can log all inference requests and responses for troubleshooting purposes, with the necessary privacy and security constraints in place. Next, the feedback taken from the agents is stored in a separate S3 bucket.

The Q&A chatbot is designed to be a multi-tenant solution and support additional health products from Amazon Health Services, such as Amazon Clinic. For example, the solution is deployed with [AWS CloudFormation](#) templates for infrastructure as a code (IaC), allowing different knowledge bases to be used.

Conclusion

This post presented the technical solution for Amazon Pharmacy generative AI customer care improvements. The solution consists of a question answering chatbot implementing the RAG design pattern on SageMaker and foundation models in SageMaker JumpStart. With this solution, customer care agents can assist patients more quickly, while providing precise, informative, and concise answers.

The architecture uses modular microservices with separate components for knowledge base preparation and loading, chatbot (instruction) logic, embedding indexing and retrieval, LLM content generation, and feedback supervision. The latter is especially important for ongoing model improvements. The foundation models in SageMaker JumpStart are used for fast experimentation with model serving being done with SageMaker endpoints. Finally, the HIPAA-compliant chatbot server is hosted on Fargate.

In summary, we saw how Amazon Pharmacy is using generative AI and AWS to improve customer care while prioritizing responsible AI principles and practices.

You can [start experimenting with foundation models in SageMaker JumpStart](#) today to find the right foundation models for your use case and start building your generative AI application on SageMaker.

About the author

Burak Gozluklu is a Principal AI/ML Specialist Solutions Architect located in Boston, MA. He helps global customers adopt AWS technologies and specifically AI/ML solutions to achieve their business objectives. Burak has a PhD in Aerospace Engineering from METU, an MS in Systems Engineering, and a post-doc in system dynamics from MIT in Cambridge, MA. Burak is passionate about yoga and meditation.

Jangwon Kim is a Sr. Applied Scientist at Amazon Health Store & Tech. He has expertise in LLM, NLP, Speech AI, and Search. Prior to joining Amazon Health, Jangwon was an applied scientist at Amazon Alexa Speech. He is based out of Los Angeles.

Alexandre Alves is a Sr. Principal Engineer at Amazon Health Services, specializing in ML, optimization, and distributed systems. He helps deliver wellness-forward health experiences.

Nirvay Kumar is a Sr. Software Dev Engineer at Amazon Health Services, leading architecture within Pharmacy Operations after many years in Fulfillment Technologies. With expertise in distributed systems, he has cultivated a growing passion for AI's potential. Nirvay channels his talents into engineering systems that solve real customer needs with creativity, care, security, and a long-term vision. When not hiking the mountains of Washington, he focuses on thoughtful design that anticipates the unexpected. Nirvay aims to build systems that withstand the test of time and serve customers' evolving needs.



Like



Share

Comments

Log in to comment
